Checkpoint A — Context, Claim and First Evidence

3-minute checkpoint. The question is: who is the user, what is the bounded problem, and what evidence could challenge the claim?

Context and claim

MatchPulse is a club-first football ticketing and fan-engagement platform. The first user is a **club organizer** running a local match. The first bounded problem is capacity: when many fans act at the same moment, the event must not sell or admit more people than the configured limit.

The first increment was capacity-safe tournament registration. The behaviour claim was:

> For a tournament organizer, the platform will keep participation at or below the configured capacity and keep point balances, positions and trades consistent under concurrent requests, measured by a concurrent burst and committed-state assertions against a real PostgreSQL database, with the limitation that the evidence is local/CI, uses one database instance, and has no real-user observation.

This is a **behaviour claim**, not an outcome claim. The problem claim — that organizers experience overbooking and value the control — is a hypothesis. The outcome claim — that the platform reduces reconciliation work — is also a hypothesis. I state both limits rather than presenting technical evidence as user validation.

First increment

An organizer creates an event with a capacity. A fan registers. The database function locks the event row, checks capacity, inserts the registration and increments the counter in one transaction. A duplicate is refused without consuming a second place. A full event tells the fan that it is full.

The architecture boundary was visible: the UI did not decide capacity. The database did.

Evidence

The test is 100 concurrent registration requests against capacity 32. It asserts the accepted/refused split **and** committed rows, the stored counter and `counter <= capacity`. The first version of this evidence was reopened because it could only pass: it checked response counts, not committed state. The corrected suite was observed failing when the capacity guard and CHECK constraints were removed (`Expected: 32, Received: 100`) and when uniqueness was removed (`Expected: 1, Received: 10`).

Finding. The original capacity guard was already correct; the real defect was uniqueness. The earlier documentation had mislabelled it. That is why fault injection matters more than a green run.

Limit and decision

The limit is one local database, one machine, authenticated clients, no production network or retry behaviour, and a planning-assumption traffic number. The capacity claim is accepted for the recorded mechanism. The next increments were the live market, authentication/RLS and club tenancy. I did not claim a security boundary until authentication and RLS existed.

Detail: invariant registry · methodology checkpoints · requirements

Checkpoint B — Operating Model and Feedback

3-minute checkpoint. Why does this lifecycle fit the main uncertainty, how does one item move from idea to use, and which feedback changes a decision?

Main uncertainty

The main uncertainty is **solution and integration uncertainty**, not whether the product category exists. The domain rules — capacity, points, resolution, club isolation — must be stable; the implementation and the evidence are discovered in slices. A fully plan-driven lifecycle would commit too early to an unvalidated product. A fully adaptive lifecycle would put the concurrency invariants at risk.

Lifecycle

I use a **hybrid lifecycle**:

Layer	Choice	What it covers
Domain contract	Plan-driven	Kata, requirements, domain model, invariants, schema constraints, ADRs, RPC contracts
Delivery	Iterative and incremental	Issue → branch → implementation → tests → PR → CI → review → merge
Scope and priority	Adaptive	Backlog order, non-goals, pivots, residual-risk decisions

Flow of one item

A unit of work is a **slice**, not a whole feature: intake (issue with scope and Done-when), plan, build (one branch), verify (invariants, fault injection, static checks, smoke), review (diff, tests, docs, security), integrate (CI plus human authorization), and learn (link evidence, update the board).

Where work waits. The main queues are CI/review, human authorization, the local Docker/Supabase environment and product decisions. In the measured cycles, PR feedback was 2m54s–3m32s, merge-to-green was 2m15s–2m39s, and WIP never exceeded one implementation PR. One 21-minute platform lag on #69 was not explained by job duration. These are one developer's cycles, not a benchmark.

Feedback that changed a decision

The important correction was early: the operating model claimed CI protected domain correctness, but CI ran only lint and build. The invariant job was added in #60. The documentation had asserted a control that did not exist; the record was corrected rather than hidden.

Fault injection demotes a test that cannot fail; review changes a lock order or adds missing evidence; flow evidence changes WIP policy; the AI log changes the AI boundary. Organizer/fan feedback and production monitoring are **missing**: there is no user study and no deployment.

Observed, proposed, unknown

- **Observed:** capacity holds under a 100-request burst; double-spend is prevented across two markets; resolution is one-shot; WIP is one slice.
- **Proposed:** the hybrid lifecycle explains the practice; it is not a controlled experiment.
- **Unknown:** whether organizers value the control; multi-instance behaviour; sustained load; the market signal's value.

If adoption uncertainty dominates, run organizer interviews before more engineering. If solution uncertainty dominates, add a spike. If the system becomes production-critical, add plan-driven security, release and recovery gates. If flow shows a persistent queue, change WIP, review ownership or slice size and measure again.

Detail: SDLC decision · flow evidence

Checkpoint C — Quality and Residual Risk

3-minute checkpoint. Which risk matters most, what behaviour did you test, what did review change or keep, and what remains uncertain?

Risk control matrix

Risk	Control	Effort	Residual risk
R1. Unauthorized access or role escalation	Supabase Auth, RLS, least-privilege grants, `SECURITY DEFINER` RPCs with `auth.uid()`, role checks, authorization suite	High	No independent security audit
R2. Oversell or over-capacity	Event/ticket row locks, capacity and allocation triggers, cumulative buyer-limit trigger, v3 tests, fault injection	High	Sales-path trigger not separately mutated; no production load
R3. Double charge or double payout	Profile/market locks, conditional debit, request-key idempotency, terminal resolution, double-spend and lifecycle tests	High	Single database instance; M-INV2 now observed-red
R4. Privacy leak in insights	Club-scoped RPCs, aggregate-only columns, outcome/time-bucket suppression, audited export, retention cleanup	High	No independent privacy review
R5. Operational failure without recovery	Health, logs, smoke, CI delivery/recovery, runbook, public-schema restore	Medium	No hosted deployment, external monitoring or production recovery (FND-006)

Non-test control: the RLS/grants boundary, architecture guard, code review, branch protection and human merge gates. R1 and R4 are not solved by tests alone.

Which risk matters most? R1. An authorization failure makes every other guarantee meaningless: a person who can act as another user can move points or resolve a market. It was the top risk until authentication and RLS existed; the boundary is tested but not independently audited.

Quality story

Behaviour. One profile with 60 points; two concurrent 50-point entries in two markets. Exactly one commits. The other is refused for insufficient points. The balance never goes negative. Exactly one trade and one position exist.

Evidence. `market-concurrency.test.ts` runs the race against a real PostgreSQL database and asserts committed state. With the profile lock and both balance guards removed, both entries were accepted, the balance reached **−40**, and the test failed (`Expected: 1, Received: 2`). Restored, the suite returned green.

Limit. One local database, two markets, no network retry. It proves the transactional rule, not production or multi-instance behaviour.

Review and AI

Review changed the `close_trade` lock order before commit: the first draft locked the position before the profile, contradicting `market → profile → cooldown → position`. Review also kept the cooldown-exit implementation and added the missing test. The v3.1.0 audit then found and fixed real defects in anonymous sign-in, legacy attendee reads, result binding, buyer limits, reward idempotency and retention.

AI contributed code, tests and docs; the human decided scope, rules and risk. AI-generated tests can share assumptions with AI-generated code, so the checks are the business-rule oracle, observed-red fault injection, CI and human gates. No third-party audit.

Residual risk and next action

FND-002 is verified: 28 fault-injection entries, including M-INV2 observed-red. FND-004: p95 is asserted in CI for the 100-registration and 20-ticket bursts; production and multi-instance load are not measured. FND-006: local/CI delivery, production deferred. FND-023 is closed by #188 (legacy analytics retired). FND-026 is accepted (simulated points); FND-027 is verified with a 7-day grace and a daily `pg_cron` job. The next evidence is a hosted recovery drill and an independent privacy review.

Detail: risk map · quality story · residual risk · findings register

Checkpoint D — Architecture and Changeability

3-minute checkpoint. The focus is ownership, boundaries, dependency rules, fitness functions, a realistic change, compatibility and technical debt.

Ownership

- **Club `OWNER` / `ADMIN`:** club settings, members, events, tickets and plans.
- **Club `OWNER` / `ADMIN` / `STAFF`:** match result and market resolution.
- **Club `ANALYST`:** aggregate insights only; no attendee identity.
- **Platform admin:** clubs, plans, subscription status and audit; no impersonation.
- **Operator:** retention, backup and recovery with the service role.
- **Human owner:** schema, RPC contracts, RLS and releases, through an approved migration.

The database is the system of record. The UI and CLI are clients; neither is the authorization boundary.

Boundaries and fitness functions

The system is one Next.js application and one PostgreSQL database. Pages and Server Actions call `src/lib/domain/`, which calls authorized RPCs or reads. The CLI uses the same contract and never imports Server Actions. Domain modules — identity, clubs/plans, events, ticketing, market, insights, admin/audit — communicate through SQL inside one transaction, not through a network.

`scripts/check-architecture.ts` guards the text-level rules: no service-role key in `src/`, no direct table mutations, no server-client imports from components, and mutation RPCs with auth checks and grants. The behavioural fitness functions are the authorization suite, the v3 tenancy suite, the CLI suite and the fault injection matrix.

A realistic change: v3 club tenancy and nominal ticketing

The v3 change is the best example. **What changed:** clubs became the tenant; events became club-owned; plans/entitlements gated features; ticketing became nominal; a shared contract and CLI were added; platform admin became separate. **Why:** the old global-organizer model could not express a club subscription, staff roles, attendee identity or club-scoped insights.

Compatibility: migrations `20261201` – `20261212` are additive. Existing organizers are backfilled into clubs with `OWNER` / `PRO`. Legacy v2 pages remain. Anonymous fan sessions are intentionally not migrated; that is a deliberate break because all users now use email/password. A rollback is a migration replay or a public-schema restore; there is no production rollback because there is no production.

What did not change: the modular monolith, the database boundary, the transaction model and the LMSR market.

Technical debt and evolution

Legacy v2 pages remain; the legacy `CLUB` role and analytics path were retired in #188. There is no hosted deployment, real payments, seat maps or turnstile integration; those are separate projects. Attendee retention is a local command. There is no independent security/privacy review.

The architecture should extract a module only when a measured driver appears: independent scaling, independent release cadence, a different data owner or a regulatory boundary. None exists today. A microservice split would add network failure modes and distributed consistency without a benefit the kata requires.

Decision. Keep the modular monolith and the database boundary. Retire the legacy surface before adding scope. Record any boundary change in an ADR.

Detail: v3 verification · architecture guard · ADR-007 · code map

Checkpoint E — Delivery, Observation and Recovery

3-minute checkpoint. The focus is how an increment reaches a running environment, how I know it is healthy, what happens when it fails, how recovery works, and what risk remains.

Delivery

Local and CI only. There is no hosted environment, TLS, DNS or production availability claim.

- **Local:** Docker + Supabase CLI; `bun install`, `bunx supabase start`, `bunx supabase db reset`, `bun run dev`. A new person can replay every migration from an empty schema.
- **CI:** five jobs — lint/typecheck/build/security, domain invariants, delivery/recovery, fault injection, E2E/accessibility.
- **Container:** `Dockerfile` and `docker-compose.yml` run the app against the host Supabase stack.

`db reset` deletes local data; it is not a production operation.

Observation

Signal	What it proves	What it cannot
`GET /api/health`	App + database reachable, no sensitive data	Business correctness, latency
JSON logs	A health or key action failure happened	Trends, alerting, correlation
`bun run smoke`	Health, login and home shell work	Authenticated purchase/trade, load
CI status	The recorded checks ran and passed	Production behaviour
Capacity and ticket burst p50/p95/max	p95 is asserted in CI for the named bursts	Production load, multi-instance

There is no collector, dashboard, trace or alert. Logs go to stdout and are lost when the process stops. That is acceptable for a course prototype and is stated as a limit.

Release and rollback

Releases are annotated SemVer tags on `main` after CI is green. v3.0.0 is the current tag; the v3.1.0 remediation is in the changelog `Unreleased` section. A rollback is a previous tag plus a migration replay or a public-schema restore. There is no production rollback because there is no production. Breaking changes are named: anonymous sessions removed, reward redemption requires a request key, resolution requires a finished match and the score-derived winner.

Recovery

The CI `Delivery and recovery` job dumps the application-owned `public` schema, restores it into a scratch database and verifies tables, RLS policies, rows and a marker. The last recorded drill restored a 67,009-byte backup: tables 10 → 10, policies 10 → 10, tournaments 2 → 2, marker present.

It proves the application schema and policies can be restored. It does **not** prove Supabase-managed auth/storage, point-in-time recovery, production RTO/RPO, encryption keys or recovery from a corrupted backup.

SLOs and limits

Proposed, not production commitments: health returns 200 in smoke; p95 < 1,000 ms for 100 concurrent registrations and 20 concurrent ticket purchases, asserted in CI (observed 878 ms and 241 ms); 0 invariant failures in the domain suite; public-schema restore matches tables/policies/rows. Availability is not claimed.

The limits are: no hosted deployment, external monitoring, alerting, on-call, production backup, retention, RTO/RPO or incident drill. The p95 is a CI gate for the named local bursts, not a production capacity claim. If a hosted environment is required, it is an explicit decision with cost, security and operational approval.

Detail: runbook · SLOs · recovery drill · p95 gate · delivery verification